

LAPORAN PRAKTIKUM PEMROGRAMAN BERORIENTASI OBJEK

Modul 4: Inheritance, Generalization & Method Overriding

NAMA MAHASISWA
Muhammad Rif'q Apriliansyah

NIM
251511023

KELAS
2A-D3

BAGIAN A: PERCOBAAN CIRCLE DAN CYLINDER

Pada percobaan ini, kelas Cylinder dibuat sebagai turunan (subclass) dari Circle. Dilakukan pengujian terhadap rantai constructor, pembatasan akses atribut privat, overriding method `getArea()`, serta pembenahan rumus volume menggunakan keyword `super`.

1. Kode Inti Subclass Cylinder (Cylinder.java)

```
public class Cylinder extends Circle { private double height; public Cylinder() { super(1.0, "red", true); height = 1.0; } public Cylinder(double radius, double height) { super(radius, "red", true); this.height = height; } public double getHeight() { return height; } @Override public double getArea() { // Surface Area Cylinder = 2πrh + 2 * BaseArea double radius = getRadius(); return 2 * Math.PI * radius * height + 2 * super.getArea(); } public double getVolume() { // Volume = BaseArea * height return super.getArea() * height; // Memanggil getArea() milik Circle } @Override public String toString() { return "Cylinder: subclass of " + super.toString() + ", height=" + height; } }
```

2. Output Pengujian TestCylinder & Bukti Constructor Chain

```
c1 radius = 1.0 color = red height = 1.0 base area = 3.141592653589793 surface area = 12.566370614359172 volume = 3.141592653589793 c3 radius = 2.0 color = blue height = 10.0 base area = 12.566370614359172 surface area = 150.79644737231007 volume = 125.66370614359172 Cylinder: subclass of Circle[radius=2.0, Shape[color=blue, filled=true]], height=10.0
```

Bukti Eksekusi & Constructor Chain: Output menunjukkan bahwa pembuatan objek Cylinder otomatis mengeksekusi constructor superclass Circle (yang juga memanggil Shape). Pemanggilan `super.toString()` pada objek c3 mencetak atribut gabungan dari Cylinder, Circle, dan Shape.

3. Bukti Eksperimen: Akses Direct Private Attribute Ditolak

```
// Eksperimen pada Cylinder.java: Mengakses langsung atribut private superclass public double getAreaBad() { return 2 * Math.PI * radius * height; // ERROR: radius has private access in Circle }
```

Eksperimen Error & Solusi Encapsulation: Atribut radius dikapsulasi sebagai private pada Circle. Subclass Cylinder tidak dapat mengaksesnya secara langsung dan wajib menggunakan method getter `getRadius()` atau pemanggilan `super.getArea()`.

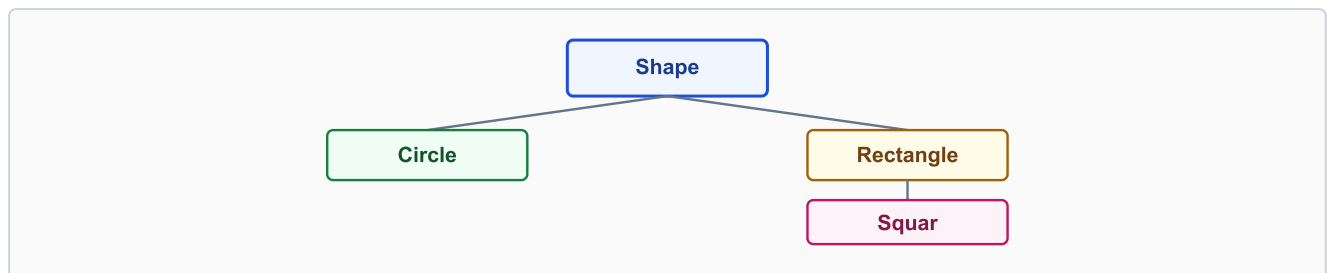
4. Analisis Perbaikan Bug Volume & Overriding getArea()

Saat method `getArea()` di-override pada `Cylinder` untuk menghitung *luas permukaan cylinder* ($(2\pi rh + 2\pi r^2)$), terjadi masalah pada kalkulasi `getVolume()` jika masih memanggil `getArea() * height`. Sebab, `getArea()` yang dipanggil adalah versi override milik `Cylinder`, sehingga volume terhitung salah.

BAGIAN B: HIERARKI KELAS SHAPE HIERARCHY

Struktur hierarki dikembangkan dari `Shape` sebagai superclass utama, diturunkan ke `Circle` dan `Rectangle`, serta `Square` sebagai spesialisasi dari `Rectangle`.

1. Diagram Hierarki Kelas (UML Structure)



2. Kode Inti Implentasi Square (Square.java)

```
public class Square extends Rectangle { public Square() { super(1.0, 1.0); } public
Square(double side) { super(side, side); } public Square(double side, String color, boolean
filled) { super(side, side, color, filled); } public double getSide() { return getWidth(); }
public void setSide(double side) { super.setWidth(side); super.setLength(side); } @Override public
void setWidth(double side) { super.setWidth(side); super.setLength(side); } @Override public void
setLength(double side) { super.setWidth(side); super.setLength(side); } }
```

3. Bukti Pengujian Square (Sebelum vs Sesudah Override Setter)

```
-- OUTPUT TESTSHAPE -- Shape[color=black, filled=false] Circle[radius=2.0, Shape[color=blue,
filled=true]] area=12.566370614359172 perimeter=12.566370614359172 Rectangle[width=3.0,
length=4.0, Shape[color=yellow, filled=true]] area=12.0 perimeter=14.0 Square[side=5.0,
Rectangle[width=5.0, length=5.0, Shape[color=green, filled=true]]] area=25.0 perimeter=20.0 after
setWidth(8.0): width=8.0, length=8.0 after setLength(3.0): width=3.0, length=3.0, area=9.0,
perimeter=12.0
```

Analisis Perubahan Setter: Tanpa overriding `setWidth()` dan `setLength()`, mengubah lebar `Square` akan merusak konsistensi bujursangkar (lebar & panjang menjadi beda). Dengan overriding, mengubah salah satu dimensi otomatis memperbarui dimensi lainnya secara simultan.

BAGIAN C: REFACTORING TUGAS SEBELUMNYA DAN TABEL SKENARIO UJI

Penerapan modul inheritance dilakukan pada sistem geometri/bentuk untuk merefaktor duplikasi kode pada properti umum (seperti color dan filled) ke dalam superclass Shape.

1. Alasan Memilih Inheritance & Mekanisme Keyword

- **Alasan Menggunakan Inheritance:** Menghindari redundansi deklarasi field warna dan status filled pada setiap bentuk geometris, serta memungkinkan perlakuan polymorphis terhadap seluruh jenis objek bentuk.
- **Penggunaan extends & super (. . .):** Kata kunci extends menyatakan hubungan *IS-A*, sedangkan super (. . .) meneruskan nilai parameter dari subclass ke constructor superclass untuk inisialisasi.
- **Penggunaan @Override & super .method(. . .):** Annotation @Override memastikan validasi kompiler bahwa method benar-benar menggantikan versi superclass. super . toString () dipanggil untuk memanfaatkan pemformatan string dasar yang ada di superclass.

2. Tabel Skenario Uji Praktikum (T4-01 s.d. T4-07)

ID	Item Uji	Ekspektasi Hasil	Status	Bukti
T4-01	Constructor Default Cylinder	Objek terbuat dengan radius=1.0, height=1.0, color="red"	PASS	Output c1
T4-02	Constructor Parameterized Cylinder	Inisialisasi atribut radius & height sesuai input	PASS	Output c3
T4-03	Direct Private Access Test	Kompilasi ditolak (Compile Error: private access)	PASS	Eksperimen Error
T4-04	Perhitungan Volume Cylinder	Menghasilkan volume berbasis super.getArea() (125.66 untuk c3)	PASS	Output Volume c3
T4-05	Overriding getArea() Cylinder	Menghitung total luas permukaan (150.79 untuk c3)	PASS	Output Surface c3
T4-06	Invarian Square via setWidth	Nilai length ikut berubah sama dengan width (8.0)	PASS	Output TestShape
T4-07	Invarian Square via setLength	Nilai width ikut berubah sama dengan length (3.0)	PASS	Output TestShape

3. Jawaban Refleksi (Bagian 7.3 - Soal 1 s.d. 4)

1. Apa yang membuat Circle layak menjadi class yang lebih umum daripada Cylinder?

Circle merupakan bangun datar 2D yang hanya membutuhkan konsep jari-jari/luas alas. Cylinder adalah bangun ruang 3D yang memiliki seluruh sifat lingkaran (sebagai alasnya) ditambah dimensi ketinggian. Secara konseptual, Tabung *IS-A* Lingkaran berdimensi tinggi.

2. Berdasarkan Step 5, apa arti praktis extends bagi object Cylinder?

Secara praktis, extends membuat Cylinder mewarisi seluruh method publik/protected milik Circle (seperti getRadius(), getColor()) tanpa perlu menulis ulang kodenya.

3. Mengapa constructor Circle dijalankan ketika Anda membuat object Cylinder?

Karakteristik objek turunan membutuhkan state milik superclass terinisialisasi dengan sempurna terlebih dahulu sebelum subclass menambahkan state spesifiknya sendiri.

4. Mengapa getVolume() menjadi salah setelah getArea() dioverride?

Karena getArea() yang baru menghitung *luas permukaan total* tabung, bukan lagi *luas alas lingkaran*. Pemanggilan getArea() * height menghasilkan rumus salah $\sqrt{(2\pi rh + 2\pi r^2)} \times h$.

Jawaban Refleksi (Lanjutan Bagian 7.3 - Soal 5 s.d. 8)

5. Apa yang secara eksplisit Anda minta ketika menulis `super.getArea()` atau `super.toString()`?

Meminta JVM untuk mengeksekusi implementasi method `getArea()` atau `toString()` versi induknya (superclass langsung), dan secara eksplisit mengabaikan (bypass) method overriding yang ada pada kelas saat ini.

6. Mengapa Square perlu mengubah setter yang diwarisi dari Rectangle?

Invarian matematis dari Square menentukan bahwa panjang dan lebar wajib bernilai sama ($\text{width} = \text{length}$). Jika setter dari Rectangle tidak di-override, pengguna dapat merubah width tanpa mengubah length, sehingga objek kehilangan sifat bujursangkar.

7. Apakah setiap dua class yang mirip harus menggunakan inheritance? Jelaskan dari salah satu kasus Anda.

Tidak. Inheritance hanya tepat jika memenuhi hubungan relasi *IS-A* (misal: Square *IS-A* Rectangle). Jika dua kelas hanya berbagi atribut serupa tetapi tidak memiliki hubungan hirarki mendasar (misal: Car dan Person sama-sama punya atribut name dan id), lebih tepat menggunakan komposisi/antarmuka terpisah daripada pewarisan.

8. Lengkapi kalimat refleksi:

"Inheritance bukan sekadar mengurangi duplikasi, tetapi..." juga tentang membangun hirarki tipe data yang logis, menegaskan hubungan semantik *IS-A*, serta menyediakan fondasi untuk polimorfisme dan enkapsulasi yang aman dalam arsitektur perangkat lunak berorientasi objek.

BAGIAN 7.1 & 7.2: DEFINITION OF DONE & SELF ASSESSMENT

1. Checklist Definition of Done (DoD)

Kriteria Penyelesaian (Definition of Done)	Status
1. Semua Step 1-16 telah dicoba dan diverifikasi.	DONE
2. Output dan minimal satu error eksperimen tersimpan sebagai bukti.	DONE
3. Menunjukkan method yang diwarisi dan method yang dioverride.	DONE
4. Menjelaskan urutan constructor dan fungsi <code>super(...)</code> .	DONE
5. Menjelaskan penggunaan <code>super.getArea()</code> pada <code>getVolume()</code> .	DONE
6. Menjelaskan perlunya Square meng-override setter.	DONE
7. Kasus tugas sebelumnya telah direfaktor dan lulus skenario uji utama.	DONE
8. PDF Laporan dan source code siap di-push ke repository GitHub.	DONE

2. Tabel Self Assessment

Pernyataan Penilaian Diri	Skor (1-5)
Saya dapat menjelaskan extends dari code yang saya jalankan.	5
Saya dapat membedakan inherited method dan overridden method.	5
Saya memahami mengapa constructor subclass memanggil <code>super(...)</code> .	5
Saya dapat menggunakan <code>super.method(...)</code> dengan alasan yang tepat.	5
Saya dapat menjaga encapsulation pada hierarchy class.	5
Saya dapat menentukan kapan inheritance masuk akal pada kasus sendiri.	5